Machine Learning with Applications 25 (2026) 100987

Contents lists available at ScienceDirect

Machine Learning with Applications

journal homepage: www.elsevier.com/locate/mlwa

Self-improving alpha mining for quantitative trading via multi-agent large language models with knowledge base accumulation

Minh-Son Vu, The-Trung Pham, Hong-Viet Tran *

Institute for Artificial Intelligence, University of Engineering and Technology, Vietnam National University, Hanoi, Viet Nam

ARTICLE INFO

Keywords:
Large language models
Quantitative trading
Alpha mining
Knowledge base
Multi-agent systems

ABSTRACT

Quantitative trading strategies depend on discovering novel alpha factors that capture market inefficiencies, yet traditional approaches rely on manual feature engineering, which is time-consuming and requires substantial domain expertise. This paper proposes an automated framework that leverages large language models (LLMs) to systematically generate, refine, and evaluate trading alpha strategies through a multi-agent pipeline comprising a WriterAgent, JudgeAgent, and BacktestEngine, organized in a nested-loop architecture. The key contribution of this work is the systematic integration of WorldQuant 101 Formulaic Alphas as a structured knowledge prior, rooted in proven formulaic patterns rather than unconstrained free-form synthesis. This design significantly narrows the search space, reduces invalid code generation, and accelerates convergence of the refinement loop. Experiments on Vietnamese stock market data demonstrate that the proposed framework consistently generates alpha factors with statistically significant Information Coefficient (IC) values and positive Sharpe ratios within a short generation time. Furthermore, the incorporation of WorldQuant 101 Formulaic Alphas consistently outperforms LLM-based generation without constraints in both alpha quality and convergence efficiency, establishing a scalable, robust, and domain-based approach to automated alpha mining.

1. Introduction

The discovery of profitable trading strategies in financial markets is fundamentally a problem of finding patterns that exploit market inefficiencies. In quantitative trading, an alpha factor (or alpha) is a quantitative signal derived from market data that generates trading returns uncorrelated with systematic market risk. Identifying novel alphas is a labor-intensive process that requires a deep understanding of market mechanics, domain expertise, and extensive experimentation with feature combinations and parameter tuning.

Traditional alpha discovery follows a manual pipeline: analysts brainstorm ideas, implement them as factors, backtest them on historical data, and iterate based on results. This loop often takes weeks to months per candidate and discards most ideas early due to poor initial performance. Recent advances in large language models (LLMs) open a new avenue for automating this process. Applying these powerful models to specialized domains such as quantitative investment brings its own set of challenges (Wang et al., 2023): a central difficulty is how to equip these models with domain-specific financial knowledge so that they can reason about alpha construction the way an experienced practitioner would, rather than producing logically flawed or practically unusable signals.

Building on the self-improvement framework introduced by Wang et al. (2024), which demonstrated that LLMs can iteratively refine trading strategies through automated feedback and backtesting, we propose an extended framework that inherits the nested-loop architecture and multi-agent design while introducing a structured knowledge prior absent from previous work. Specifically, we integrate the WorldQuant 101 Formulaic Alphas as a generation foundation, grounding LLM synthesis in proven mathematical patterns and substantially narrowing the search space compared to unconstrained free-form generation. Our core contributions are:

- **Nested Loop Architecture:** An inner loop refines code quality via iterative prompting and judge-based scoring, while an outer loop backtests refined candidates and accumulates successful strategies in a persistent knowledge base, following the design of Wang et al. (2024).
- **Multi-Agent Design:** Four specialized components (WriterAgent, JudgeAgent, ReviewerAgent, BacktestEngine) collaboratively generate, validate, and improve candidate alphas without human intervention.
- **101 Formulaic Alpha Integration:** We ground LLM generation in the WorldQuant 101 Formulaic Alphas as a structured prior,

* Corresponding author.
E-mail addresses: 23020424@vnu.edu.vn (M.-S. Vu), 23020442@vnu.edu.vn (T.-T. Pham), thviet@vnu.edu.vn (H.-V. Tran).

https://doi.org/10.1016/j.mlwa.2026.100987
Received 8 June 2026; Received in revised form 11 August 2026; Accepted 28 August 2026
Available online 5 September 2026
2666-8270/© 2026 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

enabling the WriterAgent to compose and recombine proven formulaic patterns rather than generating from scratch. This reduces the invalid code rate and accelerates convergence of the refinement loop.

- **Knowledge Retrieval Integration:** Similar past alphas retrieved from the knowledge base are provided as in-context examples to the WriterAgent, while the JudgeAgent retrieves relevant knowledge to inform its evaluation, enabling the system to improve from its own accumulated discoveries.
- **Empirical Validation on Vietnamese Market:** We validate the framework on Vietnamese stock market data, reporting IC, Sharpe ratio, and win-rate metrics, demonstrating that 101-grounded generation outperforms unconstrained LLM generation in both alpha quality and discovery speed.

2. Related work

This work is related to prior research in quantitative alpha discovery, LLM-based code generation, retrieval-augmented generation, and autonomous agent systems. We focus on the aspects of these research areas that directly motivate the proposed framework design.

2.1. Quantitative trading and alpha discovery

Systematic equity strategies are often built on factor models. The Fama–French framework developed by Fama and French (2015) and the momentum extension proposed by Carhart (1997) show that returns can be partially explained by persistent market factors. Machine learning methods have also been applied to asset pricing and statistical arbitrage, with evidence that non-linear models can capture relationships missed by classical linear approaches (Gu et al., 2020; Krauss et al., 2017).

Automated factor construction has been studied through evolutionary search and formulaic alpha design. Allen and Karjalainen (1999) use genetic algorithms to discover technical trading rules, while Kakushadze (2016) provides a widely used collection of 101 formulaic alphas. These methods reduce manual feature engineering, but alpha discovery remains iterative and vulnerable to data mining bias, backtest overfitting, and spurious signals (Bailey et al., 2014; Lopez de Prado, 2018).

2.2. LLMs for code generation and refinement

Large language models have made automatic code generation more practical. Codex and HumanEval demonstrate that transformer models can synthesize executable programs from natural language prompts (Chen et al., 2021), and AlphaCode shows strong performance on programming-contest tasks (Li et al., 2022). These systems build on a broader program synthesis literature (Gulwani, 2017).

Code generation alone is insufficient for trading strategies: code can be syntactically valid while producing unusable or biased financial signals. Iterative refinement methods address this by using feedback to improve generated outputs. RLHF aligns models with human preferences (Ouyang et al., 2022), while execution-feedback and self-feedback methods improve programs without full retraining (Austin et al., 2021; Madaan et al., 2023). The proposed system follows this direction by using a JudgeAgent for implementation-level feedback and a ReviewerAgent for metric-level feedback.

2.3. Retrieval-augmented and agentic systems

In-context learning enables LLMs to adapt from examples supplied in the prompt (Brown et al., 2020). Retrieval-Augmented Generation extends this idea by retrieving relevant external records before generation (Lewis et al., 2020). For code tasks, retrieved examples can improve consistency and reduce generation errors (Parvez et al., 2021). The developed system applies this idea to alpha mining by retrieving validated alpha records and Alpha101-style references as prompt context.

Recent agent frameworks combine LLM reasoning with tool use and multi-step workflows. Chain-of-thought prompting improves reasoning on complex tasks (Wei et al., 2022), ReAct combines reasoning traces with external actions (Yao et al., 2023), and multi-agent software engineering frameworks such as MetaGPT separate work across specialized roles (Hong et al., 2024). Our system adopts a similar separation of roles for alpha generation, code review, backtesting, and feedback.

2.4. Retrieval-augmented and agentic systems

In-context learning enables LLMs to adapt from examples supplied in the prompt (Brown et al., 2020). Retrieval-Augmented Generation extends this idea by retrieving relevant external records before generation (Lewis et al., 2020). For code tasks, retrieved examples can improve consistency and reduce generation errors (Parvez et al., 2021). The developed system applies this idea to alpha mining by retrieving validated alpha records and Alpha101-style references as prompt context. Recent agent frameworks combine LLM reasoning with tool use and multi-step workflows. Chain-of-thought prompting improves reasoning on complex tasks (Wei et al., 2022), ReAct combines reasoning traces with external actions (Yao et al., 2023), and multi-agent software engineering frameworks such as MetaGPT separate work across specialized roles (Hong et al., 2024). Our system adopts a similar separation of roles for alpha generation, code review, backtesting, and feedback. Beyond software engineering and quantitative finance, LLM-based multi-agent systems have also been explored in other domains. In healthcare, Agent Hospital models interactions among patient, nurse, and doctor agents (Li et al., 2024). In insurance underwriting, specialized agents collaboratively process property claims and support risk assessment (Sajid, 2025), while PiFlow applies multi-agent collaboration to scientific discovery (Pu et al., 2025). These studies illustrate the broader applicability of coordinated, role-specialized agent architectures across diverse domains.

2.5. LLM-based trading agents

LLM-based trading agents are an emerging research direction. Wang et al. (2024) propose a self-improving multi-agent framework for automated trading strategy discovery, combining LLM reasoning, backtesting, and persistent knowledge accumulation. Our work shares this motivation but emphasizes a concrete implementation for Vietnamese equities, retrieval from both an internal knowledge base and Alpha101 references, and an explicit split between code-quality feedback and backtest-metric feedback.

3. Methodology

Fig. 1 presents the complete self-improving alpha mining architecture and the interactions among its agents, nested refinement loops, backtesting engine, and knowledge bases.

3.1. System overview

Our system implements a hierarchical agent-based architecture designed to systematically explore the space of trading alpha strategies. The system is organized around two nested loops:

Phase I - Inner Loop: For a single trading idea, iteratively generate candidate code using WriterAgent, validate syntax, score quality using JudgeAgent, and refine via feedback. This loop produces high-quality code before backtesting.

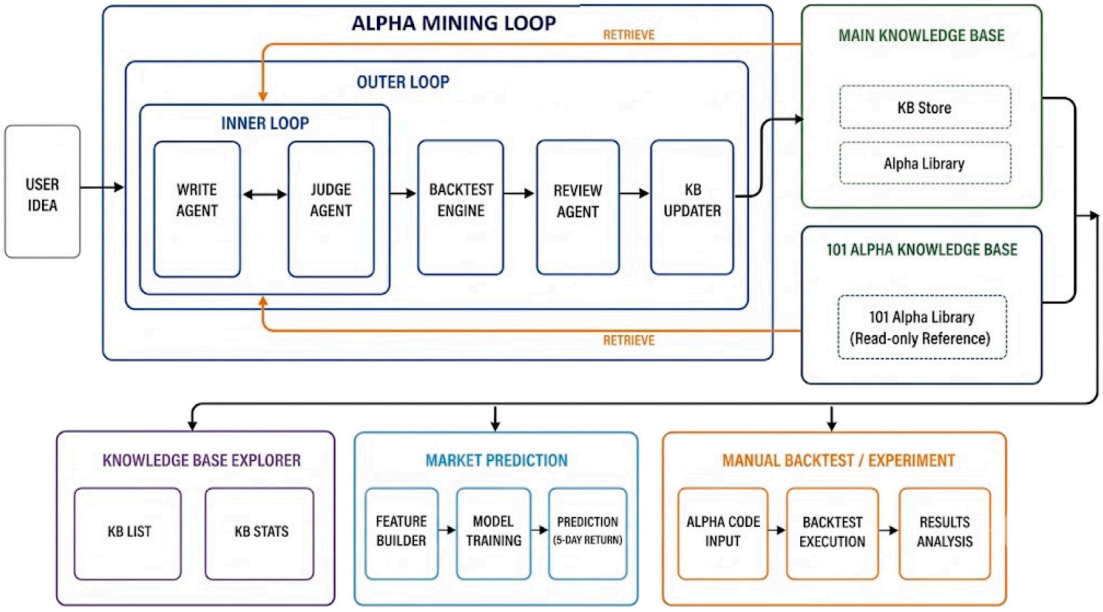


Fig. 1. Interactive self-improving alpha mining architecture.

Phase II - Outer Loop: For each trading idea, run the inner loop to obtain refined code, backtest on historical data, analyze results, and conditionally accumulate successful alphas to the knowledge base.

The **inner loop** refines candidate alpha code through iterative collaboration between WriterAgent and JudgeAgent, ensuring that only high quality candidates are backtested. The **outer loop** evaluates refined candidates on historical market data and stores successful alphas in the knowledge base. Generation uses two complementary sources: the **Main Knowledge Base**, which stores validated alphas, and the **101 Alpha Knowledge Base**, which contains the WorldQuant 101 Formulaic Alphas. At each inner loop iteration, the WriterAgent retrieves relevant examples and formulaic patterns from both sources. As validated alphas accumulate, the Main Knowledge Base provides richer context for generating subsequent trading ideas.

Each loop includes feedback mechanisms that guide both LLM and system behavior, creating a self-reinforcing cycle of discovery.

3.2. Inner loop: Iterative code refinement

The inner loop refines candidate alpha code through iterative collaboration between the WriterAgent and JudgeAgent. At each iteration, the WriterAgent retrieves similar examples from the main knowledge base and relevant formulaic patterns from the WorldQuant 101 Alpha library to ground its generation. The JudgeAgent then evaluates the candidate and returns structured feedback, which is accumulated in the context buffer for the next iteration. This process repeats until the candidate meets the quality threshold or the iteration limit is reached, as detailed in Algorithm 1.

The inner refinement loop operates on the following inputs and components:

- **Trading idea** q: A natural language description of the desired alpha signal (e.g., "high momentum stocks with low volatility"). This serves as the initial prompt seed for the WriterAgent.
- **Main Knowledge Base** Φ: A persistent store of validated alpha records accumulated across outer-loop iterations. Each record contains the original trading idea, generated code, LaTeX documentation, and backtest metrics. At each inner iteration, the WriterAgent retrieves the most similar past alpha via $C_1 = \textsc{Retrieve}(\text{Writer}, \text{ctx}, \Phi, \text{top_k}{=}1)$, providing grounded in-context examples that reduce generation errors.

Algorithm 1: The Inner Refinement Loop

Input : Trading idea q, Main Knowledge Base Φ, 101 Alpha Knowledge Base Ψ, Threshold τ, Maximum iteration limit N
Output : A refined candidate alpha α

Writer ← LLM(writer prompt)
Judge ← LLM(judge prompt)
ctx ← $\{q\}$
$n \leftarrow 0$
repeat
  $C_1 \leftarrow \textsc{Retrieve}(\text{Writer}, \text{ctx}, \Phi)$ ▷ Retrieve similar alphas from main KB
  $C_2 \leftarrow \textsc{Retrieve}(\text{Writer}, \text{ctx}, \Psi)$ ▷ Retrieve formulaic patterns from 101 Alpha KB
  ctx ← ctx ∪ $\{C_1, C_2\}$
  $\hat{\alpha} \leftarrow$ Writer.generate(ctx) ▷ Generate alpha grounded in both KBs
  $C_3 \leftarrow \textsc{Retrieve}(\text{Judge}, \text{ctx}, \Phi)$ ▷ Retrieve evaluation hints for Judge
  ctx ← ctx ∪ $\{\hat{\alpha}, C_3\}$
  score, feedback ← Judge.eval($\hat{\alpha}$, ctx) ▷ Evaluate code quality and logic
  $n \leftarrow n + 1$
  ctx ← ctx ∪ {feedback}
until score $\geq \tau \ \vee \ n \geq N$
return $\hat{\alpha}$

- **101 Alpha Knowledge Base** Ψ: A curated library of the 101 Formulaic Alphas from WorldQuant, encoded as retrievable records. The WriterAgent queries this base via $C_2 = \textsc{Retrieve}(\text{Writer}, \text{ctx}, \Psi, \text{top_k}{=}1)$ to ground generation in proven mathematical patterns, narrowing the search space compared to unconstrained free form synthesis.
- **Context buffer** ctx: A running buffer that accumulates all information available to the agents at each iteration, including the trading idea q, retrieved examples C_1 and C_2, the current candidate alpha $\hat{f}$, retrieved evaluation hints C_3, and the feedback from previous iterations. Formally, ctx is updated as ctx ← ctx ∪ $\{C_1, C_2, \hat{f}, C_3, \text{feedback}\}$ at each step.

- **WriterAgent**: An LLM prompted with a writer specific system prompt. At each iteration, it consumes the full context buffer ctx and produces a candidate alpha $\hat{f}$ in the form of executable Python code. The retrieved examples C_1 and C_2 serve as structured priors that guide the generation toward syntactically valid and financially meaningful signals.
- **JudgeAgent**: An LLM prompted with a judge-specific system prompt. It evaluates the candidate alpha $\hat{f}$ on two dimensions: code correctness (absence of look-ahead bias, valid operator usage) and logical soundness (alignment with the trading idea q). The Judge additionally retrieves evaluation hints $C_3 = \textsc{Retrieve}(\text{Judge}, \text{ctx}, \Phi, \text{top_k=1})$ from the Main Knowledge Base to inform its assessment, and returns a scalar quality score $\text{score} \in [0, 1]$ together with structured textual feedback that is appended to ctx for the next iteration.

Termination: Stop after $K_{\text{inner}} = 5$ iterations or when $s_t \geq 7.5$, whichever first. Return best code C^* and LaTeX $\mathcal{L}^*$.

3.3. Outer loop: Backtesting and knowledge accumulation

Algorithm 2: The Outer Feedback Loop

Input : Trading idea q, Other inner-loop parameters p (Main Knowledge Base Φ, 101 Alpha Knowledge Base Ψ, Threshold τ, Maximum iteration limit N), Number of outer-loop iterations K, Historical market data D
Output : Accumulated Main Knowledge Base Φ

$\Phi \leftarrow \emptyset$, $r^* \leftarrow \emptyset$, $\tau_{\text{IC}} \leftarrow 0.01$ ▷ Threshold of main KB
$\text{IC}_{\max} \leftarrow -\infty$, AlphaHistory $\leftarrow [\,]$
for $k \leftarrow 1$ **to** K **do**
  $\alpha \leftarrow \textsc{InnerLoop}(q, p)$ ▷ Refine candidate via Algorithm 1
  $\mathcal{M} \leftarrow \textsc{BacktestEngine}(\alpha, D)$ ▷ Validate on historical data
  $r \leftarrow \textsc{AlphaRecord}(q, \alpha, \mathcal{M})$
  Add r to AlphaHistory ▷ Save alpha into alpha history
end
foreach $r \in$ AlphaHistory **do**
  if $\text{IC}(r) \geq \tau_{\text{IC}}$ **then**
    if $\text{IC}(r) > \text{IC}_{\max}$ **then**
      $\text{IC}_{\max} \leftarrow \text{IC}(r)$
      $r^* \leftarrow r$
    end
  end
end
$\Phi \leftarrow \Phi \cup \{r^*\}$ ▷ Store best alpha in main KB
return Φ

The outer loop coordinates alpha discovery across multiple trading ideas and iterations. For each trading idea, the inner loop returns a refined candidate alpha that is subsequently validated by the backtest engine on historical market data. Candidates exceeding the minimum IC threshold are accumulated in the alpha history, from which the best-performing alpha is selected and stored in the Main Knowledge Base. This mechanism allows the system to learn from its own discoveries over time, progressively enriching the retrieval context available to subsequent generations, as described in Algorithm 2.

Termination: Run for up to $K_{\text{outer}} = 5$ iterations. Track the best IC achieved and return the best code, LaTeX, metrics, and history.

Data usage and selection: The alpha-mining loop does not train a supervised forecasting model and does not use a fixed temporal data partition. The inner loop only uses the trading idea, retrieved examples, previous code, and JudgeAgent feedback. The outer loop then evaluates each refined candidate on the historical backtest window D and selects the candidate with the highest IC for possible storage in the Main Knowledge Base. Therefore, the reported numbers should be read as discovery-stage backtest metrics. They are not claimed to be final blind window performance estimates.

3.4. Knowledge base retrieval

The Knowledge Base serves both as storage and as a source of in-context learning examples. For a given trading idea I, WriterAgent retrieves semantically similar past alphas using embedding based similarity:

$$C_{\text{sim}} = \text{KB.retrieve_similar}(I, \text{top_k} = 1) \tag{1}$$

The retrieval mechanism computes embeddings $\mathbf{e}$ using a sentence-transformer model (default: all-MiniLM-L6-v2) and ranks candidates by cosine similarity:

$$\text{sim}(\mathbf{e}_I, \mathbf{e}_j) = \frac{\mathbf{e}_I \cdot \mathbf{e}_j}{\|\mathbf{e}_I\| \|\mathbf{e}_j\|} \tag{2}$$

where $\mathbf{e}_I$ is the embedding of idea I, concatenated with prior successful code patterns, and $\mathbf{e}_j$ is the embedding of the jth knowledge base record.

The top-k records (default $k = 1$) are selected:

$$C_{\text{sim}} = \arg\text{-top}_k(\text{sim}(\mathbf{e}_I, \{\mathbf{e}_1, \ldots, \mathbf{e}_N\})) \tag{3}$$

Retrieved records include:

- Original trading idea and intuition
- Successful code patterns
- Achieved metrics (IC, Sharpe ratio)
- Comments from previous reviews

These examples serve as few shot demonstrations, guiding the LLM toward syntactically correct, logically sound, and financially reasonable code.

The 101 Alpha Knowledge Base Ψ is encoded as a separate retrieval source. Each record stores an identifier, required input fields, a Python expression or code fragment, and an optional natural language description. The records are embedded with the same sentence transformer model and retrieved by cosine similarity to the trading idea. In the current implementation these records are used as structural references rather than directly copied executable factors: the WriterAgent prompt explicitly asks the model to learn the pattern but not copy it verbatim, while the JudgeAgent penalizes future information usage such as `shift(-n)` and forbids forward returns inside `calc()`. The repository also contains semantic similarity utilities for detecting overlap with Alpha101 style ideas, but strict novelty scoring is not yet enforced as a hard acceptance gate.

3.5. 101 Alpha Knowledge Base

The **101 Alpha Knowledge Base** Ψ is a curated library of the 101 Formulaic Alphas from WorldQuant (Kakushadze, 2016), encoded as retrievable records and serving as a structured prior to ground the WriterAgent's generation in proven mathematical patterns. Unlike the Main Knowledge Base Φ, which accumulates validated alphas discovered during training, Ψ is static, providing an authoritative foundation of institutional grade signal patterns from the outset.

3.5.1. Record structure

Each alpha is stored as an `Alpha101Record` containing three complementary representations. The first is **exact executable code**: the complete Python implementation, directly executable to compute signals on any compatible dataset. For example, Alpha#3 is encoded as:

```
signal = -1 * open.rank(axis=1, pct=True)
             .rolling(10)
             .corr(volume.rank(axis=1, pct=True))
```

This representation enables both retrieval and direct execution within the backtesting pipeline.

The second is **abstract metadata**: automatically extracted structural fingerprints comprising the set of mathematical operators used (e.g., `rank`, `rolling`, `corr`, `diff`) and the set of lookback windows appearing in `rolling()`, `diff()`, and `shift()` calls (e.g., {5, 10, 20}). These are extracted via static analysis of the code string and appended to the record's text representation to improve retrieval precision.

The third is a **natural language description**: a plain-English summary auto generated from code tokens via a vocabulary mapping (e.g., `rolling` → "rolling window", `rank` → "cross-sectional rank"), combined with the declared input columns and extracted windows. This description serves as the primary text surface for semantic embedding.

3.5.2. Semantic retrieval

Each record is embedded using `all-MiniLM-L6-v2`, a 384-dimensional sentence encoder, with L2-normalization applied so that inner products correspond to cosine similarity. At load time, all 101 records are encoded into a fixed embedding matrix $\mathbf{E} \in \mathbb{R}^{101\times384}$. During the inner loop, the WriterAgent issues a retrieval query against Ψ at each iteration:

$$C_2 = \textsc{Retrieve}(\text{Writer}, \text{ctx}, \Psi, \text{top_k=1}) \tag{4}$$

where the query embedding $\mathbf{q} \in \mathbb{R}^{384}$ is derived from the current context buffer, and retrieval selects the record maximizing cosine similarity:

$$\hat{r} = \arg\max_{r\in\Psi} \frac{\mathbf{q}^\top \mathbf{e}_r}{\|\mathbf{q}\|\,\|\mathbf{e}_r\|} \tag{5}$$

The retrieved record, including its executable code, operators, and windows, is injected into the WriterAgent's context, grounding generation in established operator patterns and time horizons. This narrows the search space compared to unconstrained free-form synthesis and reduces the incidence of syntactically invalid or financially meaningless outputs.

3.5.3. Look-ahead verification

A critical requirement for any trading signal is *measurability*: the signal on day t must depend solely on the information available on or before t. As the 101 Formulaic Alphas originate from WorldQuant's institutional research pipeline (Kakushadze, 2016), they are designed and validated to be strictly point-in-time by construction.

3.6. Component details

3.6.1. WriterAgent

The WriterAgent is responsible for generating trading alpha code and its mathematical formulation. It takes as input a trading idea in natural language, retrieved examples from both the Main Knowledge Base and the 101 Alpha Knowledge Base, and judge feedback from previous iterations. The prompt structure is formulated as follows:

Prompt =
[System Role]+
[Task Description]+
[Trading Idea]+
[Main Knowledge Base Examples]+
[101 Alpha Knowledge Base Examples]+
[Judge Feedback]+
[Output Format Instructions]

Based on this prompt, the WriterAgent outputs Python code implementing the alpha (which inherits from `AlphaBase`) and a LaTeX formula expressing the trading logic mathematically. The agent is powered by the Llama-4-Scout-17B-16E model via the Groq API, with the generation temperature set to 0.5 to balance creativity and consistency.

3.6.2. JudgeAgent

The JudgeAgent evaluates the generated code's quality and logical correctness based on four primary evaluation criteria, each scored from 0 to 10. First, it assesses **Logic** to determine if the code correctly implements the stated trading idea. Second, it checks for **Look-ahead Bias** by ensuring the code avoids using information not available at the time of signal generation. Third, it evaluates **Robustness** in handling edge cases such as NaN values, outliers, and missing data. Finally, it measures **Clarity** to ensure the code is readable, adequately commented, and maintainable.

The final quality score is computed as an equal-weighted average of the four criteria:

$$s_t = \frac{1}{4}(\text{Logic} + \text{LookAhead} + \text{Robustness} + \text{Clarity}) \tag{6}$$

where each criterion is independently evaluated by the LLM and scored on a [0, 10] scale. The aggregated score $s_t \in [0, 10]$ determines acceptance in the inner loop. The JudgeAgent outputs this aggregated score alongside itemized feedback for each criterion. This component is also implemented using the LLaMA-3.1-8B model, but with a lower temperature of 0.2 to promote more deterministic evaluations.

3.6.3. BacktestEngine

The BacktestEngine is responsible for executing the generated alpha on historical market data and computing relevant financial metrics. Before describing the pipeline, we briefly define the core evaluation metrics. The **Information Coefficient (IC)** measures the rank correlation between generated alpha signals and realized forward returns, with higher values indicating stronger factor quality.

The **Sharpe ratio** is computed from the daily returns of an equal-weighted long-short portfolio constructed from the generated signals. The portfolio Sharpe ratio is defined as

$$\text{Sharpe} = \frac{\mathbb{E}[R_p]}{\sigma(R_p) + \epsilon}\sqrt{\frac{252}{H}}, \tag{7}$$

where R_p denotes the daily long–short portfolio return and H is the holding period (5 trading days in our experiments).

The **forward return** $\mathcal{R}_{\text{forward}}$ is the realized stock return over a fixed future horizon and serves as the evaluation target for the generated alpha signals.

The **maximum drawdown (MaxDD)** measures the largest decline from a previous peak in the cumulative portfolio value, reflecting the worst observed loss during the backtest.

Together, these metrics evaluate both the predictive quality of the alpha signals and the risk adjusted performance of the resulting trading strategy.

The pipeline execution follows a structured sequence:

1. Load historical OHLCV data for all stocks in the dataset.
2. Instantiate the generated alpha class.
3. Call alpha.calc(data) to generate raw signals $\mathcal{S}_{\text{raw}}$.
4. Apply signal normalization and NaN handling:
$$\mathcal{S} = \text{Normalize}(\mathcal{S}_{\text{raw}}) \tag{8}$$
5. Compute the forward returns:
$$\mathcal{R}_{\text{forward}} = \text{GetForwardReturns}(\text{prices}, H = 5) \tag{9}$$
6. Compute the daily Information Coefficient:
$$\text{IC}_d = \text{spearman_corr}\big(\text{rank}(\mathcal{S}_d), \text{rank}(\mathcal{R}_{d,\text{forward}})\big) \tag{10}$$
7. Construct an equal-weight long–short portfolio by buying the top 20% ranked stocks and shorting the bottom 20% ranked stocks.
8. Compute the daily portfolio return and aggregate the evaluation metrics:
$$\mathcal{M} = \{\text{IC}, \text{Sharpe}, \text{WR}, \text{MaxDD}, \text{VR}\}. \tag{11}$$

Table 1
Comprehensive dataset statistics for Vietnamese stock market.

Metric	Value
Raw stock CSV files	500
Stocks with data in experiment window	498
Eligible stocks after ≥80% coverage filter	449
Raw date range	2001-12-19 to 2025-04-04
Experiment date range	2021-01-04 to 2025-04-01
Trading days in alpha-mining window	1057
Total OHLCV records after coverage filter	456,860
Average data coverage per eligible stock	96.3%
Eligible stocks with ≥90% coverage	363 (80.7%)
Average volume (shares/day)	531.8K
Average close price (raw unit)	18.85

To ensure the validity of the backtest, several risk mitigations are implemented. Look-ahead bias is prevented by ensuring that signals generated on day d only use information available up to day d, while the corresponding portfolio returns are evaluated over the subsequent 5 trading day horizon. Survivorship bias is mitigated by including all historical stocks regardless of their current listing status. The final dates without a complete forward return horizon are excluded from evaluation. Missing values are forward-filled, and stocks with insufficient historical observations are removed before evaluation. The BacktestEngine is implemented as a deterministic evaluation module without LLM involvement, using standard numerical libraries such as `pandas` and `scipy`.

3.6.4. ReviewerAgent

The ReviewerAgent's primary responsibility is to provide human-interpretable feedback on the backtest results. It takes the generated alpha code C, the computed backtest metrics $\mathcal{M}$, and historical performance data as inputs. Based on this information, the agent conducts a concise analysis of IC direction, Sharpe consistency, and common failure modes such as constant signals or look-ahead bias. The output is a short text summary, typically in English or Vietnamese, which serves as feedback for the next outer iteration.

3.7. Alpha code interface

All generated alphas conform to the AlphaBase interface, which ensures consistent signal generation and defensive processing:

class $\mathcal{A}$ (AlphaBase):
  inputs = ["close", "volume", …]
  def calc($self$, data : Dict[str, DataFrame]) → DataFrame :
    $\mathcal{S}_{\text{raw}} = f(\text{data}[\text{"close"}, \ldots])$
    $\mathcal{S}\text{clip} = \text{clip}(\mathcal{S}\text{raw}, -5, 5)$
    $\mathcal{S} = \mathcal{S}_{\text{clip}}.fillna(0)$
    return $\mathcal{S}$

Signal Normalization: The clipping operation constrains signal magnitudes to the range $[-5, 5]$:

$$\mathcal{S}_{\text{clip}}[i,j] = \begin{cases} -5 & \text{if } \mathcal{S}_{\text{raw}}[i,j] < -5 \\ \mathcal{S}_{\text{raw}}[i,j] & \text{if } -5 \le \mathcal{S}_{\text{raw}}[i,j] \le 5 \\ 5 & \text{if } \mathcal{S}_{\text{raw}}[i,j] > 5 \end{cases} \tag{12}$$

This ensures all generated code is executable, produces the required signal matrix output, prevents extreme outliers from dominating backtest metrics, and applies consistent defensive handling across all generated alphas.

4. Experiments

4.1. Dataset and setup

4.1.1. Market data

We conduct experiments using daily Vietnamese equity market data. The initial universe consists of 500 equity symbols with open, high, low, close, and trading volume observations. The combined historical records span from 2001-12-19 to 2025-04-04, although individual stocks differ in their listing histories and data availability. For empirical evaluation, we focus on the period from 2021-01-04 to 2025-04-01, which contains 1057 observed trading days. After aligning stocks by trading dates and excluding securities with less than 80% data coverage, the final experimental universe contains 449 stocks.

Table 2
Coverage distribution of the initial 500-stock universe.

Coverage range	Stocks	Share of universe	Final sample
<50%	32	6.4%	Excluded
50%–80%	19	3.8%	Excluded
80%–90%	86	17.2%	Included
90%–95%	34	6.8%	Included
95%–99%	78	15.6%	Included
99%–100%	251	50.2%	Included
Total	**500**	**100.0%**	**449 included**

Table 3
Data quality diagnostics for the final experimental universe.

Diagnostic	Value	Interpretation
Average coverage	96.3%	High availability after the 80% filter
Average coverage gap	3.74%	Remaining gaps are imputed before evaluation
Average daily volume	490.1K shares	Broadly liquid sample, with explicit liquidity filtering
3-sigma return observations	1.76%	Extreme observations are present but limited
OHLCV field completeness	100.0%	Retained observations contain all required price-volume fields

4.1.2. Data statistics and quality

Dataset Composition:
Table 1 summarizes the composition, time span, coverage, and principal market statistics of the dataset used in the experiments.

Sample Coverage:
Table 2 reports the availability of daily observations for the initial 500-stock universe. Coverage is measured as the fraction of observed trading days for which a stock has a valid OHLCV record during the experimental period. The final universe retains stocks with at least 80% coverage, which preserves 449 stocks while excluding securities with sparse trading histories.

Data Quality:
The retained sample is sufficiently complete for cross-sectional alpha evaluation. The average coverage of the 449 eligible stocks is 96.3%, and the remaining trading-day gaps are imputed before factor evaluation to preserve time-series continuity. Since volume is central to liquidity-sensitive alphas, low activity observations are further controlled through a volume ratio screen that removes observations when current volume is below the prior 20 day average. Table 3 summarizes the main quality diagnostics used in the experiments.

Return Distribution:
Table 4 presents descriptive statistics for the alpha-mining evaluation window. Returns are expressed in percentage terms, while the moving average volume statistic is reported in shares.

4.1.3. Alpha backtest evaluation window

The alpha-generation experiments and backtests are executed on the chronological market windows corresponding to the three experimental phases. For each generated factor, signals computed on date d are aligned with 5 day forward returns, and observations near the end of

Table 4
Descriptive statistics for the 449-stock alpha-mining sample.

Variable	Unit	Mean	Std. Dev.	Min–Max
Daily log return	%	0.01	3.77	−51.08 to 33.69
5-day forward return	%	0.30	7.64	−54.00 to 114.29
20-day average volume	Shares	510.7K	1.83M	0 to 34.23M
Intraday range	%	2.45	3.69	0.00 to 50.00

Table 5
Chronological windows for alpha mining, validation, and testing.

Scope	Trading days	Date range	Use
Training (Alpha-mining)	748	2021-01-04 to 2023-12-31	Knowledge base accumulation
Validation (Ablation)	165	2024-01-01 to 2024-08-31	Hyperparameter search & Ablation studies
Testing (Out-of-sample)	144	2024-09-01 to 2025-04-01	Final factor performance validation

each window are omitted when the full forward horizon is unavailable. This design ensures that all factor evaluations are strictly point in time and look-ahead free. Table 5 summarizes the resulting evaluation windows and their corresponding uses.

All reported performance metrics are computed by aligning generated alpha signals with realized 5 day forward returns over these respective evaluation windows.

4.1.4. Preprocessing

For each stock, we compute standardized financial features that will be used by the generated alphas:

Daily Log Returns:

$$r_t = \log\left(\frac{\text{close}_t}{\text{close}_{t-1}}\right) \tag{13}$$

Forward Returns (5-day horizon):

$$r_{t,\text{forward}} = \log\left(\frac{\text{close}_{t+5}}{\text{close}_t}\right) \tag{14}$$

Volume Ratio (liquidity filter):

$$\text{vol_ratio}_t = \frac{\text{volume}_t}{\text{MA20(volume)}} \tag{15}$$

We exclude trading days where $\text{vol_ratio}_t < 1.0$ (low liquidity). This prevents the alpha from generating spurious signals on illiquid days where execution would be difficult in practice.

These standardized features—daily log returns, 5-day forward returns, and liquidity filter—support robust signal generation during the backtesting phase.

4.1.5. Model setup

The WriterAgent uses Llama-4-Scout-17B-16E, while the JudgeAgent and ReviewerAgent use LLaMA-3.1-8B, all accessed via the Groq API. Knowledge base retrieval is performed using the all-MiniLM-L6-v2 sentence embedding model.

Reproducibility notes: The source code link is provided in the title footnote, and the main model, loop, retrieval, and preprocessing parameters are listed in Appendix A.4. The deterministic parts of the pipeline include data preprocessing, generated code execution, and BacktestEngine metric computation. The LLM calls remain stochastic API calls, so exact regeneration of individual alpha code may vary across model versions even when the same prompts and temperature settings are used.

4.2. Data split and evaluation protocol

To evaluate the automated alpha mining system, we define a structured data preprocessing pipeline, establish a chronological partition of the dataset, and outline key metrics for factor performance and agent alignment.

4.2.1. Data preprocessing

Market data, consisting of open, high, low, close, and volume (OHLCV) observations, is preprocessed via the `Preprocessor` class. The preprocessing steps are executed as follows:

1. **Data Cleaning:** Observations with non-positive values (≤ 0) in the close price or trading volume are filtered out and replaced with `NaN`.
2. **Ticker and Index Alignment:** The dataframes for all tickers are reindexed using a unified set of dates and column labels to ensure cross-sectional alignment.
3. **Controlled Imputation:** Missing values are filled up to a maximum limit of 3 consecutive trading days (`ffill(limit=3)`). Gaps exceeding this limit remain as `NaN` to prevent long term data fabrication.
4. **Return Computation:** Daily returns are computed as percentage changes in close prices:

$$R_{i,t} = \frac{P_{i,t} - P_{i,t-1}}{P_{i,t-1}} \tag{16}$$

where $P_{i,t}$ is the closing price of stock i on day t. Numerical infinities resulting from computation are replaced with `NaN`.

4.2.2. Temporal data split

To mitigate selection bias and provide an unbiased evaluation protocol, the historical market data are partitioned chronologically into three non-overlapping periods corresponding to alpha discovery, model selection, and out-of-sample evaluation. This temporal split preserves the natural ordering of financial time series and prevents information leakage across different stages of the framework.

- **Training (Alpha Discovery) Period:** From `2021-01-04` to `2023-12-31` (approximately three years). During this period, the proposed framework performs iterative alpha discovery through the nested WriterAgent and JudgeAgent refinement loop and historical backtesting. Candidate alphas satisfying the predefined quality criteria are accumulated into the Main Knowledge Base, which serves as the retrieval source for subsequent generations. No performance reported in this paper is taken directly from this stage.
- **Validation (Model Selection) Period:** From `2024-01-01` to `2024-08-31` (eight months). This period is used exclusively for model selection and experimental analysis. Candidate alphas generated from the training stage are evaluated on this validation window to determine the optimal framework configuration, including the number of inner-loop iterations, outer-loop iterations, retrieval settings, and alpha acceptance threshold. Ablation studies evaluating the contributions of the Main Knowledge Base and the WorldQuant 101 Formulaic Alpha Knowledge Base are also conducted on this period.
- **Held-out Testing Period:** From `2024-09-01` to `2025-04-01` (seven months). After fixing all framework configurations based on the validation results, the final system is evaluated on a held-out test period that is never used during alpha discovery or model selection. During this phase, the Main Knowledge Base remains frozen, preventing any additional alpha accumulation or parameter adjustment. All reported out-of-sample performance metrics are obtained exclusively from this testing period.

4.2.3. Evaluation protocol and performance metrics

Factors generated by the model are backtested against the historical data. The primary target variable is the cross-sectional 5-day forward return:

$$R_{i,t,t+5} = \frac{P_{i,t+5} - P_{i,t}}{P_{i,t}} \tag{17}$$

The evaluation metrics computed by the BacktestEngine are defined as:

- **Information Coefficient (IC):** The daily cross-sectional Spearman rank correlation coefficient between the daily alpha signal and the 5-day forward return:

$$\mathrm{IC}_t = \rho_{\mathrm{Spearman}}\left(\mathrm{rank}(S_t), \mathrm{rank}(\mathcal{R}_{t,t+5})\right) \tag{18}$$

  where S_t is the cross-sectional vector of normalized alpha signals on day t.
- **Sharpe Ratio:** The annualized Sharpe ratio computed from the daily returns of an equal weight long-short portfolio formed by buying the top 20% ranked stocks and shorting the bottom 20% ranked stocks:

$$\mathrm{Sharpe} = \frac{\mathbb{E}[R_p]}{\sigma(R_p)}\sqrt{\frac{252}{5}} \tag{19}$$

  where R_p denotes the daily portfolio return.
- **Maximum Drawdown (MaxDD):** The largest decline from a previous peak in the cumulative portfolio value during the backtest.
- **Valid Ratio (VR):** The percentage of trading days for which both the Information Coefficient and the portfolio return can be successfully computed.

To verify the alignment between LLM judgment and actual factor quality, we calculate the Spearman rank correlation coefficient (ρ) and its statistical significance (p-value) between the quality scores generated by the JudgeAgent and the empirical backtest metrics (IC and Sharpe ratio) across all generated strategies during the test phase.

4.3. Experimental protocol

4.3.1. Trading ideas

We manually design 20 trading ideas, each expressing a distinct market hypothesis. Ten ideas are used for validation and ablation studies to select the best configuration, while the remaining ten are reserved for the final evaluation. Examples include "High Momentum, Low Volatility", "Mean Reversion (Overbought)", "Volume Breakout", "Price Breakout (20-MA)", "RSI Oversold + Volume", and "Bollinger Band Reversal". The aggregate results reported in this paper are computed using the 10 test trading ideas.

4.3.2. Execution

For each reported run, we execute the proposed framework configured with up to $K_{\mathrm{outer}} = 5$ outer-loop iterations. Within each outer iteration, the inner loop runs for a maximum of $K_{\mathrm{inner}} = 5$ iterations, employing an early stopping mechanism if the code quality score reaches or exceeds 7.5. The generated alphas are evaluated by the BacktestEngine on the alpha-mining backtest window, with metrics computed only on valid dates where realized 5-day forward returns are available. Alphas that achieve an IC ≥ 0.01 are successfully accumulated into the Knowledge Base. Throughout the execution, we also record the wall clock time from the initial idea input to the completion of the final backtest to evaluate the system's operational efficiency.

This design intentionally evaluates alpha discovery rather than a supervised prediction service. The same historical window is used for candidate scoring, outer-loop selection, and the reported discovery metrics. This avoids the incorrect impression that the alpha mining experiment contains a separate blind validation window, but it also means that selection effects remain possible because the highest IC candidate is selected from repeated generated trials.

Table 6
Ablation results on the validation set.

Ablation study	Configuration	Mean IC	Mean sharpe	Valid ratio (%)
Loop settings	No Loop	−0.0528	−1.93	87.5%
	Inner Only	−0.0255	−1.28	83.8%
	Outer Only	0.0122	0.59	83.5%
KB settings	No KB	0.0178	1.50	82.7%
	Main KB Only	0.0061	0.62	84.5%
	Alpha101 Only	0.0124	1.06	85.1%
Full (Proposed)		**0.0232**	**1.46**	**82.9%**

4.3.3. Baseline comparisons

To evaluate the effectiveness of our architecture, we compare our framework against several baselines. As a negative control, we use synthetically generated random alphas. We also benchmark against a manual baseline composed of simple, well known strategies, such as plain 20 day momentum. To understand the contribution of our iterative feedback mechanism and Alpha101 retrieval, we compare three LLM generation configurations: direct WriterAgent generation without feedback, the full nested loop without the 101 Alpha retrieval prior, and the full nested loop with the 101 Alpha retrieval prior. This comparison isolates the incremental value of retrieval-guided generation, but it is not a raw Alpha101 benchmark in which all original 101 formulas are executed unchanged on the Vietnamese panel.

4.4. Results

4.4.1. Ablation analysis of system components

To understand the individual contributions of the nested-loop design and the dual knowledge bases, we analyze the results of the two ablation groups using the 10 validation trading ideas described in Section 4. Table 6 summarizes the performance metrics of all configurations evaluated on these validation trading ideas.

4.4.1.1. Loop architecture ablation on validation set. We evaluate the performance of four configurations: (1) *No Loop* (direct single step generation), (2) *Inner Only* (optimizing code quality iteratively without outer loop backtest feedback), (3) *Outer Only* (running outer loop iterations without inner loop optimization), and (4) the *Full* nested loop.

Fig. 2 illustrates the results across the validation ideas. The *Full* configuration achieves the highest IC and Sharpe ratio while maintaining a high valid code ratio. Removing the inner loop (*Outer Only*) substantially reduces the valid ratio, as the WriterAgent cannot correct syntax errors before execution. Conversely, removing the outer loop (*Inner Only*) prevents validation against historical returns, leading to lower IC and Sharpe ratios. This shows that code level correction and return level feedback are complementary. Although *No Loop* has a slightly higher average valid ratio (87.5% vs. 82.9%), its simpler signals require less lookback history, while the more complex indicators from *Full* introduce warm up periods that slightly reduce validity but provide stronger predictive power.

4.4.1.2. Loop architecture ablation on test set. We further evaluate the four loop configurations on the held-out test set to examine whether the observed improvements generalize beyond the validation period. Fig. 3 shows the results across the test segments. The *Full* configuration generally achieves stronger IC and Sharpe performance than the other configurations, supporting the effectiveness of combining the inner and outer loops.

4.4.1.3. Knowledge base ablation on validation set. We compare the performance of four knowledge base configurations: (1) *No KB* (no context retrieval), (2) *Main KB Only* (using only historically validated alphas), (3) *Alpha101 Only* (using only pre-loaded formulaic patterns), and (4) the *Full* configuration (retrieving from both).

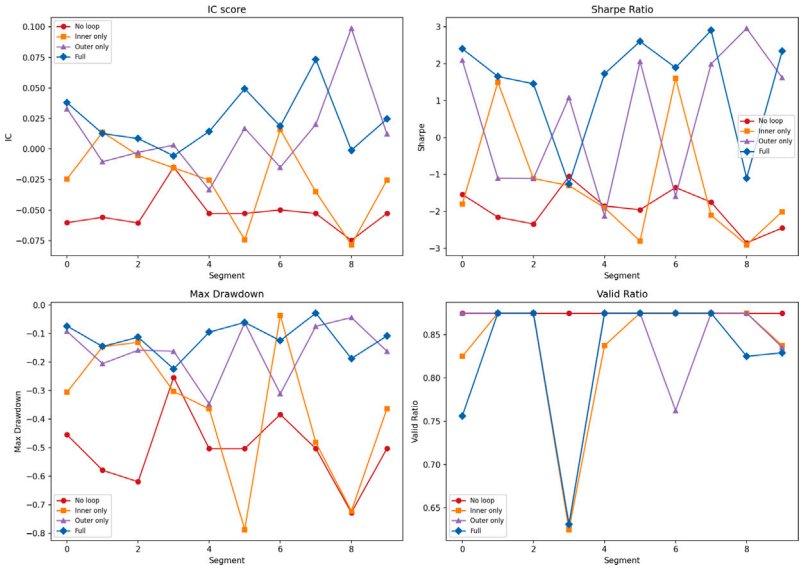


Fig. 2. Validation performance of different loop configurations.

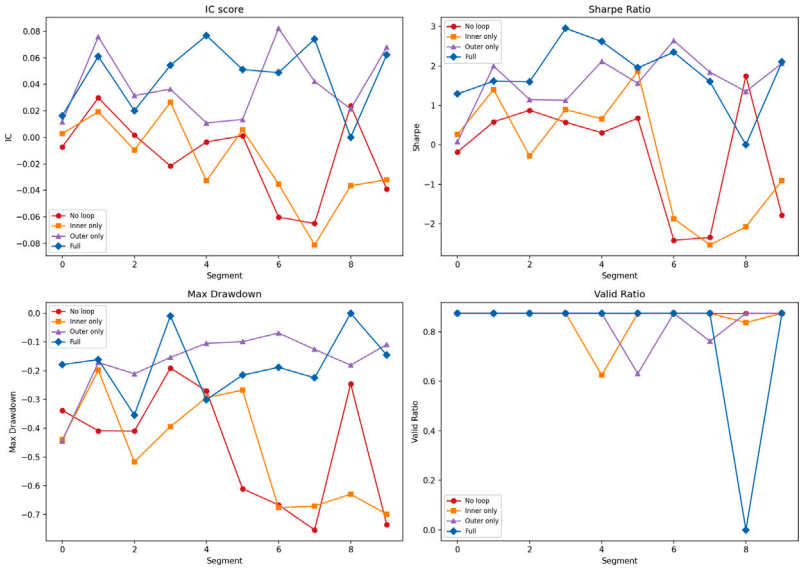


Fig. 3. Test performance of different loop configurations.

As shown in Fig. 4, the *Full* setup provides the best overall trade off. *Alpha101 Only* offers a strong mathematical prior and a high valid ratio but lower alpha ranking strength, while *Main KB Only* improves IC and Sharpe by reusing successful patterns but is more prone to syntax issues. Combining both sources allows the *Full* setup to leverage formulaic patterns and empirical discoveries, resulting in stronger and more stable performance.

4.4.1.4. Knowledge base ablation on test set. We conduct the same knowledge base ablation on the held-out test set. Fig. 5 presents the results across test segments. The *Full* configuration shows strong overall IC and Sharpe performance, indicating that combining the main knowledge base with the Alpha101 prior improves factor generation.

4.4.2. Discovered alpha performance

Table 7 reports the aggregate discovery rate and execution time across the experimental runs.

Key Observation: The outlined system discovers positive-IC alpha candidates in under 2 min on average. Manual alpha design typically requires days-to-weeks per idea, so this wall-clock result indicates a substantial reduction in candidate research time, although hardware-standardized timing studies are left for future work.

Table 7
Alpha discovery statistics across the reported experimental runs.

Metric	Value
Total ideas tested	10
Alphas with IC ≥ 0.01	9
Average time per idea	1.5 min

We report backtest metrics for representative discovered alphas in Table 8:

Interpretation: IC $\approx$ 0.0480 shows strong cross-sectional ranking quality; ICIR $\approx$ 0.4711 indicates good information coefficient consistency over time; TIC $\approx$ 5.6 reflects a solid t-statistic of information coefficient, confirming statistical significance; Sharpe $\approx$ 2.39 indicates excellent risk-adjusted consistency; Win Rate $\approx$ 65.0% is significantly above random (50%); Max DD $\approx$ 7.8% is low, indicating tight risk control; Valid Ratio $\approx$ 85.0% demonstrates strong code robustness.

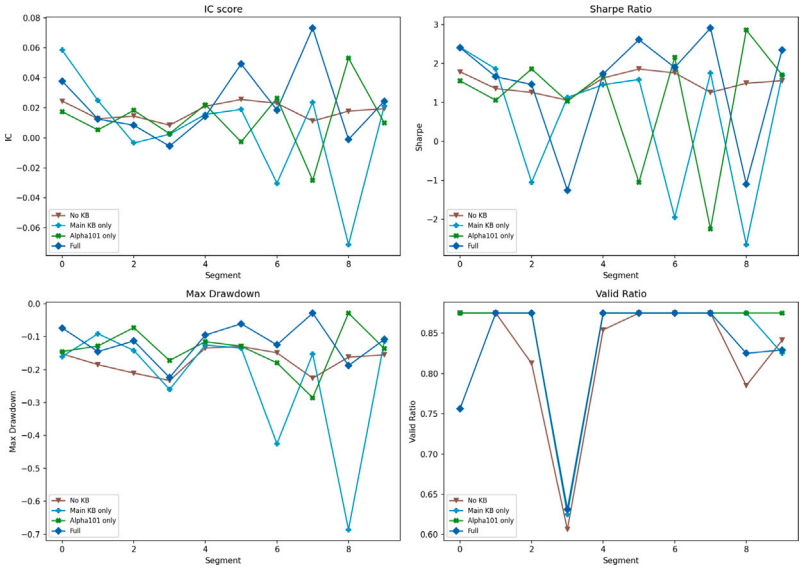


Fig. 4. Validation performance of different knowledge base settings.

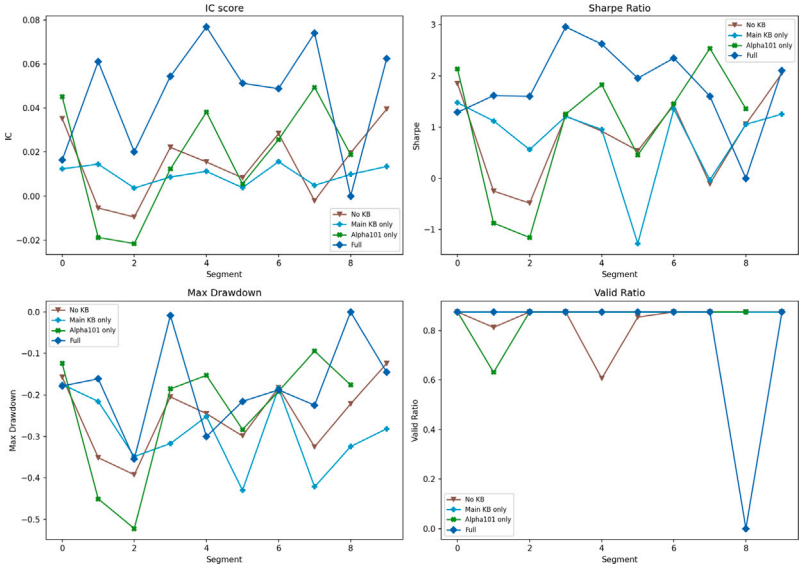


Fig. 5. Test performance of different knowledge base settings.

Table 8
Performance of representative generated alphas over the alpha-mining evaluation window.

Alpha	IC	ICIR	TIC	Sharpe	WR	MDD	VR
Price-volume divergence	0.0523	0.5448	5.9	2.64	73.1	2.0	85.6
Narrow Candle Consol.	0.0921	0.6841	8.5	3.25	79.0	4.2	85.6
Hidden Vol Accumulation	0.0812	0.8533	9.3	3.12	82.4	2.5	85.6
Decelerating downtrend	0.0517	0.5355	5.8	2.56	68.9	5.0	85.6
Repeated resistance test	0.0159	0.1086	1.1	1.72	55.9	11.4	79.9
Return persistence	0.0019	0.0162	0.2	0.95	47.1	15.5	85.6
Return dist asymmetry	0.0340	0.4244	4.6	2.15	66.4	9.1	85.6
Trend continuation	0.0576	0.5527	9.8	2.71	45.0	15.5	85.6
Wide range low liq.	0.0600	0.5624	6.1	2.78	67.2	4.7	85.6
Bullish candle dominance	0.0328	0.4292	4.7	2.05	64.7	7.7	85.6
Mean	**0.0480**	**0.4711**	**5.6**	**2.39**	**65.0**	**7.8**	**85.0**
Std	**0.0277**	**0.2491**	**3.2**	**0.66**	**12.4**	**5.0**	**1.8**

Compared to the baseline configurations on the out-of-sample test period, the proposed Alpha101-grounded framework achieves a Mean IC of 0.0480, representing an absolute increase of +0.0459 over the Direct baseline (0.0021) and +0.0168 over the standard nested-loop system (0.0312) without the formulaic prior. Similarly, its Mean Sharpe ratio of 2.39 is +2.03 points higher than the Direct baseline (0.36) and +0.20 points higher than the standard nested-loop system (2.19). This confirms that grounding the multi-agent search in structured formulaic priors significantly enhances out-of-sample factor quality and consistency.

4.4.3. Comparison with the WorldQuant 101 baseline

To verify whether the alphas generated by our proposed framework outperform the formulaic patterns they leverage as priors, we benchmark our final out-of-sample testing results against the original WorldQuant 101 Alphas. The comparison is conducted using the 10 held-out test trading ideas. Both the proposed system and the baseline are evaluated over the identical out-of-sample testing period (from 2024-09-01 to 2025-04-01). Table 9 summarizes this direct comparison.

As shown in Table 9, our proposed system consistently outperforms the original WorldQuant 101 Baseline, achieving a Mean IC

Table 9
Out-of-sample comparison with the WorldQuant 101 baseline on Vietnamese equities.

Configuration/Baseline	Mean IC	Mean sharpe	Valid ratio (%)
Original WorldQuant 101 Baseline	0.0251	1.009	73.8%
Ours	**0.0480**	**2.39**	**85.0%**

of 0.0480 compared to 0.0251 (a relative improvement of +50.5%). This demonstrates that the Agent does not merely reproduce the formulaic priors, but actively adapts and optimizes their logic to fit the specific dynamics of the Vietnamese stock market. Furthermore, our proposed system achieves a Valid Ratio of 85.0%, which is substantially higher than the 73.8% of the original formulas. This indicates that the nested inner refinement loop successfully reformulates the factors with defensive programming, significantly increasing factor robustness and executability on the target stock panel.

4.4.4. Improving performance with the iterative feedback loop

Table 10 demonstrates the effectiveness of the iterative feedback loop. Initially, generated signals often perform poorly (average IC: −0.0261, Sharpe: −0.64) due to logical errors, noise, or inverted predictions. By automatically analyzing these results and refining the code, the agent significantly improves the strategies. The final optimized signals achieve an average IC of 0.0480 and a Sharpe ratio of 2.39—a massive gain of +0.0741 in IC and +3.03 in Sharpe.

Overall, this comparison clearly proves that the feedback loop is highly effective. The agent can successfully learn from its own mistakes and turn weak initial ideas into profitable trading strategies.

4.4.5. Inter-run consistency analysis

To evaluate the consistency of the proposed framework, we conducted five independent runs for each of the ten trading ideas under the same settings. The generated alpha candidates varied across runs, but the final performance remained relatively stable. Table 11 summarizes the IC, Sharpe ratio, and Valid Ratio results across all iterations.

The results show consistent performance across independent runs. The Valid Ratio is also highly stable, indicating that the framework produces similar evaluation outcomes despite the stochastic nature of LLM generation.

4.5. Role of multi-agent architecture

The four-agent design enables separation of concerns:

- **WriterAgent:** Pure generation (no judgment of quality).
- **JudgeAgent:** Automated code review (before execution).
- **BacktestEngine:** Domain specific validation (financial metrics).
- **ReviewerAgent:** High level analysis and suggestions.

Each agent can be improved independently (e.g., swap WriterAgent model, refine JudgeAgent criteria, add new BacktestEngine metrics). This modularity is key to the framework's extensibility.

5. Discussion

5.1. Interpretation of results

Our experiments demonstrate that our approach successfully discovers statistically meaningful trading alphas across a diverse set of trading ideas. The results in Table 8 reveal several notable findings. Two alphas, Narrow Candle Consol. and Hidden Vol Accumulation, achieve particularly strong performance with IC values of 0.0921 and 0.0812 and Sharpe ratios of 3.25 and 3.12 respectively, placing them in the excellent range by quantitative finance standards. Win rates for these strategies reach up to 82.4%, indicating highly consistent directional ranking ability. The remaining alphas produce IC values between 0.01 and 0.05 and Sharpe ratios between 1.6 and 2.6, which remain practically useful when combined with diversification and portfolio construction.

We highlight three broader implications of these results:

- **Practical Significance:** IC values in the 0.01–0.05 range are meaningful for alpha research when combined with diversification and disciplined portfolio construction. The consistently positive Sharpe ratios across all discovered alphas suggest that the generated signals carry genuine and stable factor information rather than noise, but transaction cost and turnover analysis is still required before making deployment claims.
- **Benchmark Comparison:** Published academic research on alpha discovery commonly reports IC values in the 0.01–0.02 range (Grinold, 1989). Our framework produces alphas competitive with peer-reviewed standards, with several strategies substantially exceeding this benchmark.
- **Cost Efficiency:** Discovering alphas of this quality within minutes via automated LLM generation, compared to weeks of manual research, represents a significant gain in research productivity across the entire set of evaluated trading ideas.

5.2. Comparison with recent agentic alpha-mining systems

Recent agentic and evolutionary alpha-mining frameworks pursue similar automation goals but make different design choices. QuantAgent, discussed in Section 2, established the two-layer inner/outer loop that this work builds upon, but grounds generation solely in a self-accumulated knowledge base without an external formulaic prior (Wang et al., 2024). AlphaForge instead abandons LLM-based reasoning altogether, mining formulaic factors through a generative-predictive neural network trained to maximize IC, and dynamically recombining the resulting factor zoo through a separate composite model (Shi et al., 2024). FactorEngine treats factor discovery as program-level mining and separates logic revision, parameter optimization, and local computation, with knowledge-infused bootstrapping and trajectory experience used to improve both IC stability and portfolio-level metrics (Lin et al., 2026). QuantaAlpha emphasizes trajectory-level evolution: complete mining runs are mutated and recombined, while semantic consistency between hypothesis, factor expression, and executable code is explicitly controlled (Han et al., 2026). QRAFTI is broader than alpha mining alone, focusing on agentic workflows for empirical quantitative-finance research, including data abstractions, tool use, and auditable research traces (Lim et al., 2026).

Table 12 summarizes the key characteristics of these five systems alongside the proposed framework, highlighting the central methodological distinction: the integration of a static, externally-validated formulaic prior (WorldQuant 101) as a retrieval source running in parallel with a self-accumulating knowledge base.

The implemented framework occupies a lighter-weight point in this design space. Instead of building a full evolutionary search engine or a general empirical research platform, it uses a compact Writer–Judge–Reviewer loop and grounds generation with retrieved Alpha101 formulas plus previously validated alphas. The expected advantage is lower engineering complexity, faster iteration, and easier adaptation to a market with limited public factor infrastructure such as Vietnamese equities. The expected trade-off is that our pipeline currently provides weaker search governance than trajectory-level evolutionary systems: it does not yet perform novelty scoring, parameter optimization, turnover-aware filtering, multiple testing correction, or strict final blind window validation. Consequently, the present results should be interpreted as evidence that Alpha101-grounded retrieval improves practical alpha generation, rather than as a claim of empirical superiority over FactorEngine, QuantaAlpha, QRAFTI.

Table 10
Performance comparison between the first and best iterations in alpha mining.

Alpha	First iteration			Best iteration (Final)			Improvement	
	IC	ICIR	Sharpe	IC	ICIR	Sharpe	Δ IC	Δ Sharpe
Price-volume divergence	−0.0008	−0.0070	−0.11	0.0523	0.5448	2.64	+0.0531	+2.75
Narrow Candle Consol.	−0.0322	−0.2630	−0.85	0.0921	0.6841	3.25	+0.1243	+4.10
Hidden Vol Accumulation	−0.0141	−0.1370	−0.45	0.0812	0.8533	3.12	+0.0953	+3.57
Decelerating downtrend	−0.0262	−0.2880	−0.80	0.0517	0.5355	2.56	+0.0779	+3.36
Repeated resistance test	0.0095	0.0720	0.85	0.0159	0.1086	1.72	+0.0064	+0.87
Return persistence	−0.0018	−0.0160	−0.15	0.0019	0.0162	0.95	+0.0037	+1.10
Return dist asymmetry	−0.0417	−0.4580	−1.25	0.0340	0.4244	2.15	+0.0757	+3.40
Trend continuation	−0.0877	−0.8890	−1.80	0.0576	0.5527	2.71	+0.1453	+4.51
Wide range low liq.	−0.0411	−0.3870	−1.15	0.0600	0.5624	2.78	+0.1011	+3.93
Bullish candle dominance	−0.0248	−0.2130	−0.65	0.0328	0.4292	2.05	+0.0576	+2.70
Average	**−0.0261**	**−0.2586**	**−0.64**	**0.0480**	**0.4711**	**2.39**	**+0.0741**	**+3.03**

Table 11
Inter-run consistency across ten trading ideas and five independent runs.

Trading idea	Run 1			Run 2			Run 3			Run 4			Run 5		
	IC	SR	VR	IC	SR	VR	IC	SR	VR	IC	SR	VR	IC	SR	VR
Price-volume divergence	0.0164	1.29	87.5%	0.0611	1.61	87.5%	0.0201	1.60	87.5%	0.0543	2.95	87.5%	0.0768	2.62	87.5%
Narrow Candle Consol.	0.0512	1.15	85.6%	0.0495	1.10	85.6%	0.0521	1.21	85.6%	0.0488	1.05	85.6%	0.0503	1.12	85.6%
Hidden Vol Accumulation	0.0412	1.45	85.6%	0.0405	1.38	85.6%	0.0421	1.51	85.6%	0.0418	1.46	85.6%	0.0410	1.43	85.6%
Decelerating downtrend	−0.0152	−0.45	85.6%	−0.0241	−0.85	85.6%	−0.0125	−0.31	85.6%	−0.0285	−1.05	85.6%	−0.0185	−0.65	85.6%
Repeated resistance test	0.0075	0.35	85.6%	0.0081	0.41	85.6%	0.0095	0.49	85.6%	0.0076	0.36	85.6%	0.0088	0.44	85.6%
Return persistence	0.0254	1.25	85.6%	0.0385	1.58	85.6%	0.0437	1.73	85.6%	0.0521	1.95	85.6%	0.0485	1.81	85.6%
Return dist asymmetry	−0.0215	−0.85	85.6%	−0.0195	−0.75	85.6%	−0.0221	−0.91	85.6%	−0.0185	−0.68	85.6%	−0.0205	−0.82	85.6%
Trend continuation	0.0315	1.15	85.6%	0.0285	0.99	85.6%	0.0321	1.21	85.6%	0.0295	1.05	85.6%	0.0305	1.12	85.6%
Wide range low liq.	0.0115	0.41	85.6%	0.0085	0.32	85.6%	0.0121	0.45	85.6%	0.0095	0.35	85.6%	0.0105	0.39	85.6%
Bullish candle dominance	0.0515	1.81	85.6%	0.0485	1.69	85.6%	0.0521	1.92	85.6%	0.0495	1.75	85.6%	0.0505	1.79	85.6%

Table 12
Comparison with recent LLM-driven and evolutionary alpha-mining frameworks.

Study	Key characteristics	Key difference from our system
QuantAgent (Wang et al., 2024)	Two layer inner and outer LLM loop with a self accumulated knowledge base built from the agent's own real world feedback.	Relies only on a self accumulated knowledge base without an external formula library. Our system also uses the WorldQuant 101 formula library as prior knowledge.
AlphaForge (Shi et al., 2024)	Two stage generative predictive neural network that mines and dynamically combines formulaic factors through surrogate model training.	A purely neural pipeline without LLM reasoning or a symbolic prior. Our system uses an LLM writer that reasons over natural language trading ideas and retrieved formulaic patterns.
QuantaAlpha (Han et al., 2026)	Evolutionary framework that mutates and recombines entire mining *trajectories* while preserving semantic consistency across hypotheses, expressions, and code.	Optimizes complete trajectories without an external formula library. Our system optimizes individual candidates through a Writer Judge loop guided by the WorldQuant 101 formulas.
FactorEngine (Lin et al., 2026)	Program level factor search that separates logic revision from parameter optimization and uses knowledge from financial reports.	Uses unstructured financial reports as the knowledge source. Our system uses a structured mathematical formula library, leading to higher first iteration syntactic validity.
QRAFTI (Lim et al., 2026)	General purpose multi agent research framework that orchestrates external tools to reproduce factors and generate research reports.	A general research assistant rather than a dedicated alpha generation framework. Our system is designed specifically for iterative alpha generation and refinement.
Ours	**Nested Writer Judge Reviewer loop supported by two knowledge sources: a self accumulated knowledge base and the WorldQuant 101 formula library.**	**Integrates a mathematical formula library with a self accumulated knowledge base for iterative alpha generation on the Vietnamese stock market.**

5.2.1. Comparison with an external alpha mining baseline

Beyond the internal ablation baselines, we further evaluate the proposed framework against AlphaForge (Shi et al., 2024), a reproducible non-LLM framework for automated formulaic alpha mining. AlphaForge employs a generative-predictive neural architecture to discover a set of candidate factors and dynamically combine them.

For a fair comparison, AlphaForge is trained on the same Vietnamese equity data available during the alpha-discovery period, and its discovered factors are evaluated on the same held-out test period as the proposed framework. Both methods use the same five-trading-day forward-return target, daily cross-sectional Spearman IC, and equal-weight long–short portfolio constructed from the top and bottom 20% of ranked stocks (see Table 13).

Under the common evaluation protocol, the proposed framework achieves higher IC, ICIR, TIC, Sharpe ratio, win rate, and valid ratio than AlphaForge, while also exhibiting a substantially lower maximum drawdown. In particular, the proposed framework improves IC from 0.0337 to 0.0480 and Sharpe ratio from 1.35 to 2.39, while reducing maximum drawdown from 25.7% to 7.8%. These results indicate that, under the Vietnamese equity dataset and evaluation protocol

Table 13
Mean out of sample comparison between AlphaForge and the proposed framework across the held out trading ideas in Appendix A.1.2 under the same Vietnamese equity dataset and evaluation protocol.

Method	IC	ICIR	TIC	Sharpe	WR (%)	MDD (%)	VR (%)
AlphaForge	0.0337	0.3199	3.80	1.35	58.0	25.7	82.7
Ours	**0.0480**	**0.4711**	**5.60**	**2.39**	**65.0**	**7.8**	**85.0**

considered in this study, the proposed framework achieves stronger predictive and risk-adjusted performance than the reproduced AlphaForge baseline.

5.3. Why some ideas failed

Analysis of the unsuccessful ideas encountered during our initial exploratory runs reveals common failure modes:

1. **Look-ahead Bias (2 cases):** Code inadvertently used future information (e.g., using closing price before market open). JudgeAgent caught syntax, but logic errors persisted.
2. **Constant Signals (2 cases):** Generated code always produced the same signal (e.g., always buy), making IC undefined or negative.
3. **Insufficient Feature Coverage (2 cases):** Ideas like "Sector-Momentum" assumed sector data availability; when missing, signals became all-NaN.

These failures highlight the importance of domain constraints (available data, timing rules) and suggest that providing more structured templates for WriterAgent could improve success rate.

5.4. Limitations

This study has several limitations. First, experiments are conducted only on Vietnamese equities, so cross-market generalization remains untested. Second, although candidate selection and final reporting are now separated into a held-out test window (Section 4), the outer loop still selects among a bounded number of repeated generation trials for each idea, so some residual selection effect within a run cannot be fully ruled out without full rolling-window re-estimation. Third, the evaluation focuses on factor-quality metrics such as IC and IC Sharpe, without transaction costs, turnover, capacity, or portfolio-level net returns. Finally, robustness checks such as rolling-regime analysis and sensitivity to normalization or imputation choices are left for future work.

5.5. Future directions

Future work will strengthen the evaluation protocol by incorporating multiple-testing controls and full rolling-window (walk-forward) re-estimation across successive train/validation/test boundaries, extending the single held-out split used in this study. We also plan to compare the proposed framework with operator-grammar baselines on the same Vietnamese equity dataset. To provide a more comprehensive portfolio-level evaluation, future work will include turnover-aware candidate selection, allowing the outer optimization loop to penalize high-turnover signals rather than optimizing IC alone. We also plan to perform transaction-cost sensitivity analysis by reporting net Sharpe ratios under representative trading-cost assumptions for the Vietnamese equity market (e.g., 15, 30, and 50 bps per round trip), and to estimate strategy capacity using average daily trading volume together with bounded market participation assumptions. Finally, incorporating novelty scoring and evaluating the framework across multiple equity markets would further improve its robustness and practical applicability for quantitative research.

6. Conclusion

This paper presented an automated framework for discovering quantitative trading alpha strategies through the combination of large language models and a nested feedback loop architecture. The framework integrates a WriterAgent for candidate alpha generation, a JudgeAgent for quality evaluation, and a BacktestEngine for historical validation, organized in an inner refinement loop and an outer accumulation loop that together form a self-improving discovery cycle. A demonstration chatbot system developed based on the proposed framework is publicly available at: https://github.com/VMSSON345/QuantTrading-Agent

The key contribution of this work is the integration of the WorldQuant 101 Formulaic Alphas as a structured knowledge prior, grounding LLM generation in proven mathematical patterns rather than unconstrained free-form synthesis. Within the reported discovery-stage comparison on Vietnamese stock market data, this design outperforms both the direct LLM baseline and the standard nested-loop system across IC, Sharpe ratio, and evaluation verdict metrics.

Specifically, the proposed framework achieves the following:

- **Alpha Quality:** Several discovered alphas achieve discovery-stage IC values exceeding 0.08 and Sharpe ratios above 3.0, substantially surpassing the direct generation baseline.
- **Consistency:** Positive Sharpe ratios and win rates above 50% are observed across all evaluated trading ideas, indicating that the generated signals carry genuine factor information.
- **Knowledge Base Benefit:** Dual retrieval from the Main Knowledge Base and the 101 Alpha Knowledge Base consistently improves generation quality and accelerates convergence of the inner refinement loop.
- **Efficiency:** The framework discovers viable alpha candidates within minutes, representing a significant reduction in research time compared to manual alpha development.

These results establish the developed framework as a scalable and domain-grounded approach to automated alpha mining. The self-improving nature of the Knowledge Base creates a long-term advantage: as more alphas are discovered, subsequent generation improves. Natural extensions to multi-market settings, larger base models, transaction cost modeling, and ensemble alpha combination are left for future work.

CRediT authorship contribution statement

Minh-Son Vu: Conceptualization, Methodology, Software, Data curation, Formal analysis, Investigation, Validation, Writing – original draft. **The-Trung Pham:** Methodology, Validation, Investigation, Visualization, Writing – review & editing. **Hong-Viet Tran:** Supervision, Project administration, Resources, Funding acquisition, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was supported by research project QG.26.08 of Vietnam National University, Hanoi.

Appendix

A.1. Twenty trading ideas

The twenty manually designed trading ideas are divided into two disjoint groups. The first ten ideas are used during the validation phase for model selection and ablation studies, while the remaining ten ideas are reserved for held-out testing.

A.1.1. Validation trading ideas

1. **Volatility Compression:** Intraday trading ranges (high–low) contract progressively over multiple consecutive sessions, suggesting volatility compression before a significant price movement.
2. **Daily Range Breakout:** The closing price moves outside the previous day's high–low range, indicating a potential breakout.
3. **Opening–Closing Convergence:** The distance between the current opening price and the previous closing price gradually decreases, reflecting increasing market indecision.
4. **Low Volatility Regime:** The recent 10-day volatility is substantially lower than the preceding 60-day volatility, suggesting that a volatility expansion may follow.
5. **Early Session Volume Accumulation:** The proportion of trading volume occurring early in the trading session increases over time, indicating earlier capital inflows.
6. **Ascending Lows Accumulation:** Consecutively higher daily lows are formed while the closing price remains below the previous peak, suggesting accumulation before a breakout.
7. **Volume Dispersion Anomaly:** The standard deviation of trading volume increases significantly while price remains relatively unchanged, indicating abnormal capital flow.
8. **Bullish Session Dominance:** The number of positive-return trading days within a recent window substantially exceeds the number of negative-return days, indicating persistent buying pressure.
9. **Intraday Buying Strength:** Closing prices consistently remain between the opening price and the daily high, indicating sustained intraday buying pressure.
10. **Market Correlation Decoupling:** The correlation between an individual stock's returns and the market return gradually weakens, suggesting idiosyncratic price behavior.

A.1.2. Held-out testing trading ideas

11. **Price–Volume Divergence:** Price reaches a new high while trading volume is lower than at the previous peak, indicating divergence between price appreciation and capital flow.
12. **Narrow Candlestick Consolidation:** Consecutive candlesticks exhibit very small real bodies, suggesting market consolidation before an information-driven breakout.
13. **Hidden Volume Accumulation:** The 5-day average trading volume significantly exceeds the 20-day average while price remains sideways, indicating silent accumulation.
14. **Decelerating Downtrend:** The magnitude of daily price declines gradually decreases despite an ongoing downtrend, suggesting weakening selling pressure.
15. **Repeated Resistance Test:** Price repeatedly approaches but fails to close above a resistance level across multiple sessions, indicating repeated resistance testing before a possible breakout.
16. **Short-Term Return Persistence:** Daily returns exhibit positive serial dependence with returns from one or two previous trading days, suggesting short-term momentum.
17. **Return Distribution Asymmetry:** The recent return distribution becomes strongly skewed toward gains or losses, indicating latent asymmetric risk.
18. **Trend Continuation after Pullback:** During an uptrend, price experiences a short pullback before recovering and breaking above the previous high, suggesting trend continuation.
19. **Wide Range Low Liquidity:** Exceptionally wide daily trading ranges occur together with below-average trading volume, suggesting thin-market conditions or temporary price manipulation.
20. **Bullish Candlestick Dominance:** The proportion of bullish candlesticks within a recent window is significantly higher than expected given the stock's historical volatility, indicating persistent buying pressure.

A.2. Sample generated alpha code

This section presents two representative alpha implementations generated by our system.

A.2.1. Alpha 1: Price-volume divergence/momentum reversal

```python
from alpha_base import AlphaBase
import numpy as np
import pandas as pd

class MomentumReversal(AlphaBase):
    """
    High momentum (20-day price change) combined with
    volume surge predicts 5-day reversal.
    """
    inputs = ['close', 'volume']

    def calc(self, data):
        close = data['close']
        volume = data['volume']

        # 20-day momentum
        momentum = close.pct_change(20)

        # 20-day volume MA
        vol_ma = volume.rolling(20).mean()
        vol_ratio = volume / vol_ma

        # Signal: high momentum + volume surge
        signal = momentum * np.log1p(vol_ratio)

        # Clip and fill NaN
        signal = signal.clip(-5, 5)
        signal = signal.fillna(0)

        return signal
```

A.2.2. Alpha 2: Bollinger Band Reversal (IC = 0.0112)

```python
from alpha_base import AlphaBase
import numpy as np

class BollingerReversal(AlphaBase):
    """
    When price touches lower Bollinger Band,
    expect reversion over next 5 days.
    """
    inputs = ['close']

    def calc(self, data):
        close = data['close']

        # 20-day Bollinger Bands
        ma20 = close.rolling(20).mean()
        std20 = close.rolling(20).std()
        lower_band = ma20 - 2 * std20

        # Distance from lower band (normalized)
        distance = (close - lower_band) / std20

        # Inverted signal: near lower band -> negative
        signal = -distance.clip(-5, 5)
        signal = signal.fillna(0)

        return signal
```

A.3. WriterAgent prompt template

The WriterAgent is guided by a structured prompt template designed to enforce code correctness and financial validity. The system prompt establishes the agent role and output format, while the user prompt is populated at runtime with retrieved context from both knowledge bases and feedback from the previous iteration.

Table A.1
Model configuration used by the designed framework components.

Component	Configuration
WriterAgent model	Llama-4-Scout-17B-16E (Groq API)
WriterAgent temperature	0.5
WriterAgent max tokens	2048
JudgeAgent model	LLaMA-3.1-8B (Groq API)
JudgeAgent temperature	0.2
JudgeAgent max tokens	1024
ReviewerAgent model	LLaMA-3.1-8B (Groq API)
ReviewerAgent temperature	0.3
ReviewerAgent max tokens	512
KB embedding model	all-MiniLM-L6-v2

WriterAgent Prompt Template

```
[SYSTEM]
You are a Quantitative Analyst specializing in
    the Vietnamese
stock market. Write an AlphaFactor from a given
    trading idea.

OUTPUT FORMAT:
  1. LaTeX formula
  2. Complete Python implementation

ABSOLUTE RULES:
  - No shift(-n) with n > 0  (look-ahead bias
    forbidden)
  - No forward returns inside calc()
  - Signal must be continuous float, not binary
  - All operations must be vectorized
    (pandas/numpy)
  - Apply rank(axis=1, pct=True) - 0.5 at final
    step

[USER]
Trading idea    : {idea}
KB context      : {kb_context}
Alpha101 context: {alpha101_context}
Feedback        : {feedback_section}

REQUIREMENTS:
  - Follow the trading idea closely
  - Learn patterns from KB but do not copy
  - Reference WorldQuant alpha structure but do
    not reproduce
  - Incorporate feedback from previous
    iteration if provided
```

The placeholders `{kb_context}` and `{alpha101_context}` are populated at runtime with examples retrieved from the main knowledge base Φ and the WorldQuant 101 Alpha library Ψ respectively. The `{feedback_section}` is empty on the first iteration and populated with JudgeAgent feedback on subsequent iterations.

A.4. Experimental configuration

A.4.1. Model configuration
See Table A.1.

A.4.2. Refinement and backtesting configuration
See Table A.2.

A.4.3. Data processing configuration
See Table A.3.

Table A.2
Refinement loop and backtesting configuration.

Parameter	Value
Inner loop iterations	$K_{\text{inner}} = 5$
Inner loop quality threshold	$\theta_{\text{inner}} = 7.5/10$
Outer loop iterations	$K_{\text{outer}} = 5$
Outer loop IC threshold	$\theta_{\text{save}} = 0.01$
Signal clipping range	$[-5, 5]$
KB retrieval Top-K	1
Forward return horizon	5 days
Liquidity filter threshold	vol_ratio ≥ 1.0

Table A.3
Data preprocessing and sample filtering configuration.

Component	Configuration
Missing value handling	Reindex with forward/back fill
Data coverage minimum	80%
Alpha evaluation period	2021-01-04–2025-04-01
Number of stocks	449 eligible (500 initial symbols)
Total trading days	1057
Total data points	456,860

Data availability

Data will be made available on request.

References

Allen, F., & Karjalainen, R. (1999). Using genetic algorithms to find technical trading rules. *Journal of Financial Economics*, *51*(2), 245–271.

Austin, J., Odena, A., Nye, M., et al. (2021). Program synthesis with large language models. arXiv preprint arXiv:2108.07732.

Bailey, D. H., Borwein, J. M., Lopez de Prado, M., & Zhu, J. (2014). Pseudo-mathematics and financial charlatanism: The effects of backtest overfitting on out-of-sample performance. *Notices of the American Mathematical Society*, *61*(5), 458–471.

Brown, T. B., Mann, B., Ryder, N., et al. (2020). Language models are few-shot learners. *Vol. 33*, In *Advances in neural information processing systems*.

Carhart, M. M. (1997). On persistence in mutual fund performance. *The Journal of Finance*, *52*(1), 57–82.

Chen, M., Tworek, J., Jun, H., et al. (2021). Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374.

Fama, E. F., & French, K. R. (2015). A five-factor asset pricing model. *Journal of Financial Economics*, *116*(1), 1–22.

Grinold, R. C. (1989). The fundamental law of active management. *The Journal of Portfolio Management*, *15*(3), 30–37.

Gu, S., Kelly, B., & Xiu, D. (2020). Empirical asset pricing via machine learning. *The Review of Financial Studies*, *33*(5), 2223–2273.

Gulwani, S. (2017). Program synthesis. In *Formal methods and software engineering* (pp. 1–15).

Han, J., Zhang, S., Li, W., Dong, Y., Hu, T., Zhu, Y., Yu, X., Guo, X., Liu, Z., Wang, K., Liu, J., Jiang, T., An, R., Hu, S., Yang, Z., Che, R., & Wang, H. (2026). QuantaAlpha: An evolutionary framework for LLM-driven alpha mining. arXiv preprint arXiv:2602.07085.

Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Wang, J., Zhang, C., Wang, Z., Yau, S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., & Schmidhuber, J. (2024). MetaGPT: Meta programming for a multi-agent collaborative framework. In *International conference on learning representations*.

Kakushadze, Z. (2016). 101 formulaic alphas. *Wilmott*, *2016*, 72–81.

Krauss, C., Do, X. A., & Huck, N. (2017). Deep neural networks, gradient-boosted trees, random forests: Statistical arbitrage on the S&P 500. *European Journal of Operational Research*, *259*(2), 689–702.

Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in neural information processing systems* (pp. 9459–9474).

Li, Y., Choi, D., Chung, J., et al. (2022). Competition-level code generation with AlphaCode. *Science*, *378*(6624), 1092–1097.

Li, J., Lai, Y., Li, W., Ren, J., Zhang, M., Kang, X., Wang, S., Li, P., Zhang, Y.-Q., Ma, W., & Liu, Y. (2024). Agent hospital: A simulacrum of hospital with evolvable medical agents. arXiv preprint arXiv:2405.02957.

Lim, T., Muthuraman, K., & Sury, M. (2026). QRAFTI: An agentic framework for empirical research in quantitative finance. arXiv preprint arXiv:2604.18500.

Lin, Q., Feng, R., Feng, Y., Huang, Z., Chen, Y., Yang, Z., Zhou, L., Fei, B., Liu, J., & Li, Y. (2026). FactorEngine: A program-level knowledge-infused factor mining framework for quantitative investment. arXiv preprint arXiv:2603.16365.

Lopez de Prado, M. (2018). *Advances in financial machine learning*. Wiley.

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Gupta, S., Majumder, B. P., Hermann, K., Welleck, S., Yazdanbakhsh, A., & Clark, P. (2023). Self-Refine: Iterative refinement with self-feedback. In *Advances in neural information processing systems*.

Ouyang, L., Wu, J., Jiang, X., et al. (2022). Training language models to follow instructions with human feedback. arXiv preprint arXiv:2203.02155.

Parvez, M. R., Ahmad, W. U., Chakraborty, S., Ray, B., & Chang, K.-W. (2021). Retrieval augmented code generation and summarization. arXiv preprint arXiv:2108.11601.

Pu, Y., Lin, T., & Chen, H. (2025). Piflow: Principle-aware scientific discovery with multi-agent collaboration. arXiv preprint arXiv:2505.15047.

Sajid, M. I. (2025). Multi-agentic automation for evaluating property claims in underwriting. *Open Journal of Applied Sciences, 15*(4), 819–833.

Shi, H., Song, W., Zhang, X., Shi, J., Luo, C., Ao, X., Arian, H., & Seco, L. (2024). AlphaForge: A framework to mine and dynamically combine formulaic alpha factors. arXiv preprint arXiv:2406.18394.

Wang, S., Yuan, H., Ni, L. M., & Guo, J. (2024). QuantAgent: Seeking holy grail in trading by self-improving large language model. arXiv preprint arXiv:2402.03755.

Wang, S., Yuan, H., Zhou, L., Ni, L. M., Shum, H.-Y., & Guo, J. (2023). Alpha-GPT: Human-AI interactive alpha mining for quantitative investment. arXiv preprint arXiv:2308.00016.

Wei, J., Wang, X., Schuurmans, D., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. arXiv preprint arXiv:2201.11903.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *International conference on learning representations*.



Minh-Son Vu is a member of the NLP Lab at the Institute for Artificial Intelligence, University of Engineering and Technology, Vietnam National University, Hanoi, Vietnam. His research interests include machine learning, large language models, multi-agent systems, quantitative finance, and AI-driven decision support systems. His recent work focuses on autonomous agents, alpha mining, and the application of large language models in financial analysis.



The-Trung Pham is a member of the NLP Lab at the Institute for Artificial Intelligence, University of Engineering and Technology, Vietnam National University, Hanoi, Vietnam. He is interested in machine learning, large language models, retrieval-augmented generation (RAG), multi-agent systems, and quantitative finance. He has worked on research projects involving autonomous agents, alpha mining, and applying LLMs and RAG to finance.



Hong-Viet Tran received the Ph.D. degree in Computer Science and is currently a researcher at the Institute for Artificial Intelligence. His research interests primarily lie in natural language processing (NLP) and its applications across various domains such as finance, healthcare, and scientific information systems. His recent work focuses on leveraging large language models (LLMs) for domain-specific reasoning, knowledge extraction, and intelligent decision support systems.